testing-mt-others

Ninad Chaudhari

February 2026

0.0.1 Specification Language

A metamorphic relation for the NL-mediated setting is a first-order logic formula over system states in Σ, utterances in $\mathcal{U}$, the observation function $\mathcal{O}$, the mediator $\mathcal{M}$, the semantic predicates Syn, $Refine$, and $Indep_\sigma$, set-theoretic comparisons $\{\subseteq, \supseteq, =\}$ over control sets $2^{\mathcal{C}}$, cardinality comparisons $\{<, \leq, =, \geq, >\}$ over data states, Boolean connectives $\wedge, \vee, \neg, \Rightarrow$, and universal quantification $\forall \sigma, u$ with the natural interpretation. We assume formulas are given in prenex normal form. Using this language, the relation

> *semantically equivalent utterances under identical conditions must activate identical controls*

is expressed as:

$$MR_{SE} : \forall \sigma \in \Sigma, \forall u_1, u_2 \in \mathcal{U}.\ Syn(u_1, u_2) \implies \mathcal{O}(\sigma \xrightarrow{u_1} \sigma') = \mathcal{O}(\sigma \xrightarrow{u_2} \sigma'')$$

1 Formal Model

1.1 Application Control Surface

We consider web applications whose graphical user interface exposes a finite, discoverable control surface. Every interactive element—visible toggles, dropdown selections, checkbox filters, menu actions, and programmatically accessible but visually hidden parameters—is a member of a universal control set

$$\mathcal{C} = \{c_1, c_2, \ldots, c_n\}, \quad |\mathcal{C}| < \infty.$$

This finiteness assumption is what distinguishes our scope from fully generative UI systems: the interface already exists, and the natural language channel is a *mediator* that translates user utterances into selections over $\mathcal{C}$, not a synthesizer of novel widgets.

1.2 System State and Observation

A **system state** is a tuple

$$\sigma = \langle V, H, P, D \rangle \in \Sigma$$

where:

- $V \subseteq \mathcal{C}$ is the **Framework State Vector**: the set of controls currently active in the frontend framework.
- H is the **Effective Context State**: the contextual content available to the LLM at inference time, including prior dialogue turns, summarized interaction history, persisted memory, and retrieved session artifacts.
- P is the **Prompt and Runtime Configuration**: the application-supplied system prompt, tool schemas, orchestration settings, summarization strategy, and related configuration that govern how the LLM interprets H and any incoming utterance.
- D is the **Data State**: the content or result set currently rendered in the interface (e.g., a filtered product list, computed cell values, displayed records).

An **observation function** projects the full state to its externally visible components:

$$\mathcal{O}(\sigma) = \langle V, D \rangle.$$

Two states are **observationally equivalent**, written $\sigma_1 \approx \sigma_2$, if and only if $\mathcal{O}(\sigma_1) = \mathcal{O}(\sigma_2)$.

1.3 Transition Semantics

A user utterance u triggers a state transition mediated by the LLM. We decompose this into two stages that separate the concerns of language mediation from application logic:

$$\underbrace{u}_{\text{NL utterance}} \xrightarrow{\mathcal{M}} \underbrace{V' \subseteq \mathcal{C}}_{\text{control activation}} \xrightarrow{\mathcal{A}} \underbrace{D'}_{\text{data / display state}}$$

- $\mathcal{M}$, the **LLM Mediator**, maps the current state and utterance to a new control activation: $\mathcal{M}(\sigma, u) = V'$.
- $\mathcal{A}$, the **Application Logic**, computes the data state from the active controls: $\mathcal{A}(V') = D'$.

The composite **state transition function** is:

$$\sigma \xrightarrow{u} \sigma' = \langle\, V',\ H \oplus \langle u, V', D' \rangle,\ P,\ D' \,\rangle \tag{1}$$

where $\oplus$ is a context-append operator that adds the utterance–response pair to the interaction history. A sequence of utterances $\tau = u_1 \cdot u_2 \cdots u_k$ induces a **trace**:

$$\sigma_0 \xrightarrow{u_1} \sigma_1 \xrightarrow{u_2} \cdots \xrightarrow{u_k} \sigma_k.$$

The set of reachable states from an initial state σ_0 is $\mathit{Reach}(\sigma_0) = \{\sigma \mid \exists\, \tau.\ \sigma_0 \xrightarrow{\tau} \sigma\}$.

Scope. Traditional GUI testing validates $\mathcal{A}$: given the correct control activations, does the application produce the correct output? Our metamorphic relations validate $\mathcal{M}$: given a natural language utterance, does the LLM mediator activate the correct controls?

1.4 Semantic Predicates

The metamorphic relations below rely on three predicates over utterances. We give each an operational definition to keep the framework tool-agnostic yet concrete.

Semantic Synonymy. Two utterances are synonymous when an embedding-based similarity oracle judges them as paraphrases:

$$\mathit{Syn}(u_1, u_2) \iff \mathrm{sim}\big(\mathrm{enc}(u_1),\ \mathrm{enc}(u_2)\big) \geq \theta$$

where $\mathrm{enc}(\cdot)$ is a sentence encoder and θ is a calibrated similarity threshold.

Constraint Refinement. Utterance u' *refines* u when it is constructed by appending an additional constraint clause δ that restricts the result set without contradicting u:

$$\mathit{Refine}(u', u) \iff u' = u \oplus \delta, \quad \delta \text{ specifies an additional filter over } \mathcal{C}.$$

Independence. Two utterances are *independent* in state σ when their **control footprints**—the sets of controls they toggle—are disjoint. Define the footprint:

$$\mathrm{fp}(u, \sigma) = \mathcal{M}(\sigma, u) \mathbin{\triangle} \sigma.V$$

where $\triangle$ denotes symmetric difference. Then:

$$\mathit{Indep}_\sigma(a, b) \iff \mathrm{fp}(a, \sigma) \cap \mathrm{fp}(b, \sigma) = \emptyset.$$

This mirrors the standard definition from Mazurkiewicz trace theory: two actions commute if and only if they act on disjoint alphabets [**?**].

1.5 First-Order Metamorphic Relations

Because the input space of natural language is effectively unbounded, constructing explicit and exhaustive test oracles is intractable. Instead, we define metamorphic relations (MRs) over the transition system. Let $\sigma \equiv_{V,H,P} \sigma'$ denote that two states share the same framework state vector, effective context, and runtime configuration.

Table 1: Metamorphic Relations for NL-Mediated Control Verification

Relation	**Formal Statement**	**Fault Detected**
MR_{SE} Semantic Equivalence	$\forall\, \sigma,\, u_1, u_2.\ Syn(u_1, u_2) \implies \mathcal{O}(\sigma \xrightarrow{u_1} \sigma') = \mathcal{O}(\sigma \xrightarrow{u_2} \sigma'')$	Synonym Blindness: semantically identical requests produce different control activations.
MR_{AC} Additive Constraining	$\forall\, \sigma,\, u, u'.\quad Refine(u', u) \implies \mathcal{M}(\sigma, u) \subseteq \mathcal{M}(\sigma, u') \ \wedge\ D' \subseteq D$	Constraint Leakage: adding a filter *increases* the result set or drops controls.
MR_{CA} Commuting Actions	$\forall\, \sigma_0,\, a, b.\quad Indep_{\sigma_0}(a, b) \implies \mathcal{O}(\sigma_0 \xrightarrow{a}\xrightarrow{b} \sigma_f) = \mathcal{O}(\sigma_0 \xrightarrow{b}\xrightarrow{a} \sigma'_f)$	Order Sensitivity: independent actions yield different states depending on execution order.
MR_{REV} Action Reversibility	$\forall\, \sigma_0,\, a.\ \mathcal{O}(\sigma_0 \xrightarrow{a}\xrightarrow{a^{-1}} \sigma_f) = \mathcal{O}(\sigma_0)$	Irreversibility: an undo/inverse action fails to restore the original state.

1.6 Fault Model

Each metamorphic relation is designed to surface a specific class of defect in the LLM mediator $\mathcal{M}$. Table 2 enumerates the fault classes, their formal characterizations, and the MR that detects them.

1.7 Test Generation Strategy

Each metamorphic test case is constructed by the following procedure:

Table 2: Fault Taxonomy for NL-Mediated Interfaces

Fault Class	**Characterization**	**Detected by**
Synonym Blindness	$\exists\, u_1, u_2.\ Syn(u_1, u_2) \ \wedge\ \mathcal{O}(\sigma \xrightarrow{u_1} \sigma') \neq \mathcal{O}(\sigma \xrightarrow{u_2} \sigma'')$	MR_{SE}
Constraint Leakage	$Refine(u', u) \ \wedge\ \lvert D'_{u'}\rvert > \lvert D'_u\rvert$ (adding a constraint increases results)	MR_{AC}
Order Sensitivity	$Indep_\sigma(a, b) \ \wedge\ \mathcal{O}(\sigma \xrightarrow{a}\xrightarrow{b} \sigma_f) \neq \mathcal{O}(\sigma \xrightarrow{b}\xrightarrow{a} \sigma'_f)$	MR_{CA}
Irreversibility	$\mathcal{O}(\sigma_0 \xrightarrow{a}\xrightarrow{a^{-1}} \sigma_f) \neq \mathcal{O}(\sigma_0)$	MR_{REV}
Context Amnesia	$\sigma_0 \xrightarrow{a} \sigma_1 \xrightarrow{b} \sigma_2$ where b does not affect the controls activated by a, yet $\mathrm{fp}(a, \sigma_0) \not\subseteq \sigma_2.V$	MR_{CA}

1. **Fix the initial state.** Record $\sigma_0 = \langle V_0, H_0, P_0, D_0\rangle$ by resetting the application to a known configuration.
2. **Select an MR and generate utterance variants.** For a chosen relation MR_i, produce a *source* utterance u and one or more *follow-up* utterances u' that satisfy the relation's precondition (e.g., $Syn(u, u')$ for MR_{SE}).
3. **Execute and observe.** From σ_0, execute each utterance (or utterance sequence) and record the resulting observations $\mathcal{O}(\sigma')$.
4. **Check the relation.** Verify the postcondition (set equality, subset inclusion, or state restoration) specified by the MR. Any violation is flagged as a potential fault and classified according to Table 2.

Because the LLM mediator may exhibit non-deterministic behavior (due to sampling temperature or context window effects), each test case should be repeated N times. A relation is considered *violated* if the failure rate exceeds a tolerance threshold ϵ, distinguishing genuine faults from stochastic noise:

$$\text{violation} \iff \frac{|\{i : \text{postcondition fails in trial } i\}|}{N} > \epsilon.$$

1.8 NL-Mediated Coverage

The finiteness of $\mathcal{C}$ enables a notion of **control coverage** that is unique to the NL-mediated setting. We define:

$$Cov_{NL}(U) = \frac{\left| \bigcup_{u \in U} \mathrm{fp}(u, \sigma_0) \right|}{|\mathcal{C}|}$$

where U is the set of test utterances. This metric captures the fraction of the application's control surface that is reachable via natural language. A low Cov_{NL} value indicates that significant portions of the application's functionality are inaccessible through the NL interface—a usability fault even if no MR is violated. "‘

—

Key changes from your original:

— Aspect — Before — After — ———————— — State — Single tuple x, one-shot $\mathcal{F}$ — Full state σ with transition semantics $\xrightarrow{u}$ — — Output space — Only $V' \subseteq \mathcal{C}$ — Observation function $\mathcal{O}(\sigma) = \langle V, D \rangle$ — — Separation of concerns — LLM + app logic mixed in $\mathcal{F}$ — Explicit split: $\mathcal{M}$ (mediator) vs $\mathcal{A}$ (app logic) — — Predicates Syn, $Refine$, $Indep$ — Undefined — Operationally defined with formulas — — Composition $\circ$ — Informally described — Trace semantics with $H\oplus$ context evolution — — Fault model — Absent — Table with 5 fault classes mapped to MRs — — Non-determinism — Ignored — Statistical tolerance threshold ϵ — — Coverage metric — Absent — Cov_{NL} over finite $\mathcal{C}$ —

2 Test Generation Algorithm

Algorithm 1 describes the metamorphic test generation procedure. For each selected metamorphic relation, the algorithm resets the application to a known initial state, generates utterance variants satisfying the relation's precondition, executes each variant through the NL-mediated interface, and checks the post-condition over the resulting observations. To account for LLM non-determinism, each utterance pair is evaluated N times, and a violation is reported only when the failure rate exceeds a tolerance threshold ϵ.

The CHECK function implements the postcondition specific to each MR: set equality for MR_{SE} and MR_{CA}, subset inclusion for MR_{AC}, and state restoration for MR_{REV}. The utterance generator $\mathcal{G}$ may employ paraphrase models (for MR_{SE}), constraint concatenation templates (for MR_{AC}), permutation of action sequences (for MR_{CA}), or antonym substitution (for MR_{REV}).

Input: Application under test $\mathcal{A}$, MR set $\mathcal{R}$, utterance generator $\mathcal{G}$, repetitions N, tolerance ϵ, timeout T
Output: Test report $\mathcal{T}$

$\mathcal{T} \leftarrow \emptyset$
for each $MR_i \in \mathcal{R}$ **do**
  while time() $< T$ **do**
    $\sigma_0 \leftarrow$ ResetApp($\mathcal{A}$) {known initial state}
    $(u, u') \leftarrow \mathcal{G}(MR_i)$ {source and follow-up utterances}
    $fails \leftarrow 0$
    for $j = 1$ **to** N **do**
      $\sigma_0' \leftarrow$ ResetApp($\mathcal{A}$)
      $o_1 \leftarrow \mathcal{O}(\sigma_0' \xrightarrow{u} \sigma')$
      $\sigma_0'' \leftarrow$ ResetApp($\mathcal{A}$)
      $o_2 \leftarrow \mathcal{O}(\sigma_0'' \xrightarrow{u'} \sigma'')$
      if $\neg$ Check($MR_i,\ o_1,\ o_2$) **then**
        $fails \leftarrow fails + 1$
      end if
    end for
    if $fails\,/\,N > \epsilon$ **then**
      $\mathcal{T}$.Add($MR_i,\ u,\ u'$, 'fail', $fails/N$)
    else
      $\mathcal{T}$.Add($MR_i,\ u,\ u'$, 'pass', $fails/N$)
    end if
  end while
end for
return $\mathcal{T}$ =0

Figure 1: Metamorphic Test Generation for NL-Mediated Interfaces